Sec. 12.2. CAPA

CAPA

Task 1 - Introduction

One of the challenges when analyzing potentially malicious software is that we risk our machine or environment being compromised when we run it unless we have a sandbox or a completely isolated environment where we can test all we want. Generally speaking, there are two types of analysis: dynamic analysis and static analysis. This room will focus on conducting static analysis using a tool called CAPA.

CAPA (Common Analysis Platform for Artifacts) is a tool developed by the FireEye Mandiant team. It is designed to **identify the capabilities** present in executable files like Portable Executables (PE), ELF binaries, .NET modules, shellcode, and even sandbox reports. It does so by analyzing the file and applying a set of rules that **describe common behaviours**, allowing it to determine what the **program is capable of doing**, such as **network communication**, **file manipulation**, **process injection**, and many more.

The beauty of CAPA is that it encapsulates years of reverse engineering knowledge into an automated tool, making it accessible even to those who may not be experts in reverse engineering. This can be incredibly useful for analysts and security professionals, allowing them to quickly understand potentially malicious software's functionality without manually reverse engineering the code.

This tool is particularly useful in malware analysis and threat hunting, where understanding a binary's capabilities is crucial for incident response and defensive measures.

Task 2 - Tool Overview: How CAPA Works

First, open a PowerShell, noting that it might take a while before the prompt appears. Next is to make sure that you are in the correct directory (`C:\Users\Administrator\Desktop\capa`); then you need to run `capa` or `capa.exe`, then point to the binary, and that's it!

In this example, we will use **cryptbot.bin;** please note that the results of this file will be discussed throughout the succeeding task. After running the command, wait for the result, which may take several minutes. **We don't intend this to finish but rather to let you get a feel for running the tool**, so we suggest that you continue the task while CAPA is running or stop the processing. There are alternative ways to proceed with the analysis of the results. See the command below.

```
PS C:\Users\Administrator\Desktop\capa> capa.exe .\cryptbot.bin
loading : 100%|████████████████████| 485/485 [00:00<00:00, 1108.84     rules/s]
/ analyzing program...
```

In addition to the `-h` command, which gives us more information about the parameters available with the tool, we will use two (2) most used parameters, which is the `-v` and `-vv`, which will give us a more detailed result. However, this will increase the processing time. We will have a discussion about the results of these options in the coming tasks.

Option	Description	Sample Syntax
`-h` or `--help`	Show this help message and exit.	`capa -h`
`-v` or `--verbose`	Enable verbose result document.	`capa.exe .\cryptbot.bin -v`
`-vv` or `--vverbose`	Enable a very verbose result document.	`capa.exe .\cryptbot.bin -vv`

This should be the output of the command. **Please take note that results may vary. If you ran CAPA on some files, it might or might not give the same information as below.*

Since we know that the processing time takes several minutes, we have pre-processed this to a file named **cryptbot.txt** located in **C:\Users\Administrator\Desktop\capa .** Open another PowerShell terminal and use the command

```
Get-Content cryptbot.txt
```

```
PS C:\Users\Administrator\Desktop\capa> Get-Content .\cryptbot.txt
```

Loading the content to PowerShell will give the same output from the terminal above.

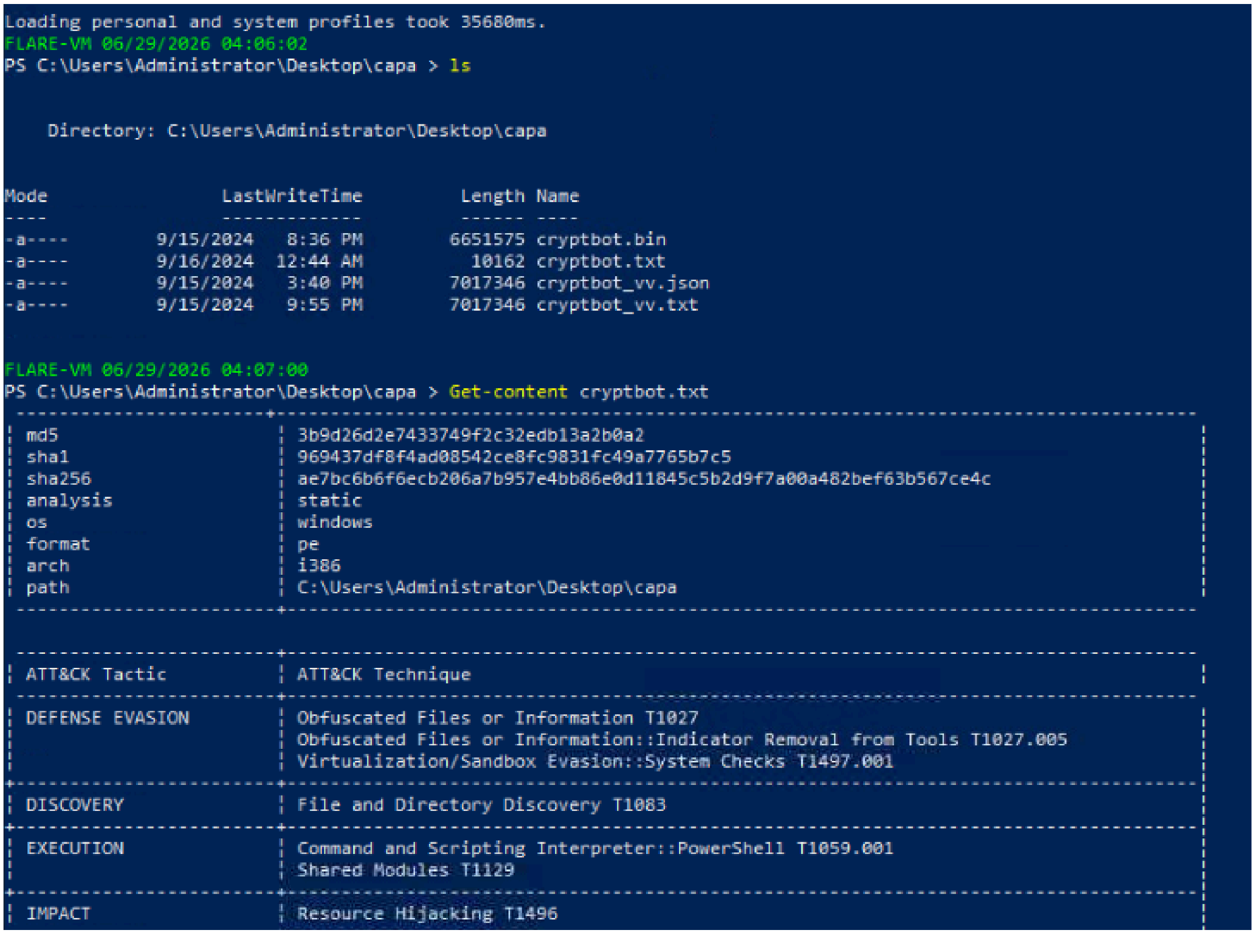
Answer the questions below

> What command-line option would you use if you need to check what other parameters you can use with the tool? Use the shortest format. -h

> What command-line options are used to find detailed information on the malware's capabilities? Use the shortest format. v

> What command-line options do you use to find very verbose information about the malware's capabilities? Use the shortest format. vv

> What PowerShell command will you use to read the content of a file? Get-Content



Task 3 - Dissecting CAPA Results Part 1: General Information, MITRE and MAEC

As mentioned in the previous task, the results of running CAPA against cryptbot.bin will be discussed in the succeeding task. As such, we will dissect the results per block and topic. The first block contains basic information about the file. This includes the following:

- The cryptographic algorithms, such as the `md5`, and `sha1/256`.
- The `analysis` field tells us how CAPA performed its analysis on the file.
- The `os` field reveals the operating system (OS) context for which the identified capabilities apply.
- The `arch` field allows us to determine whether we are dealing with a binary related to x86 architecture.
- The `path` where the analyzed file was located.

```
┌──────────────┬──────────────────────────────────────────────────────────────────
──────────────┐
│ md5          │ 3b9d26d2e7433749f2c32edb13a2b0a2
│
│ sha1         │ 969437df8f4ad08542ce8fc9831fc49a7765b7c5
│
│ sha256       │ ae7bc6b6f6ecb206a7b957e4bb86e0d11845c5b2d9f7a00a482bef63b567ce4c
│
```

```
| analysis    | static
|
| os          | windows
|
| format      | pe
|
| arch        | i386
|
| path        | /home/strategos/Room-CAPA/cryptbot.bin
|
└─────────────┴──────────────────────────────────────────────────────────────────────────────
──────────────┘
```

MITRE ATT&CK: The MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) framework is a comprehensive global knowledge repository that meticulously documents the tactics and techniques employed by threat actors at every stage of a cyber-attack. It functions as a strategic playbook, providing detailed insights into attackers' methods, from **gaining initial access** to **maintaining a presence**, **escalating privileges**, **evading defenses**, **moving laterally within a network**, and much more. CAPA uses this format for the output. Note that some results may or may not contain the Technique and Sub-technique Identifier.

Format	Sample	Explanation
ATT&CK Tactic::**ATT&CK Technique**::**Technique Identifier**	Defense Evasion::Obfuscated Files or Information::T1027	**DEFENSE EVASION =** ATT&CK Tactic**Obfuscated Files or Information =** ATT&CK Technique**T1027 =** Technique Identifier
ATT&CK Tactic::**ATT&CK Technique**::**ATT&CK Sub-Technique**::**Technique Identifier**[.]Sub-technique **Identifier**	Defense Evasion::Obfuscated Files or Information::Indicator Removal from Tools T1027.005	**DEFENSE EVASION** = ATT&CK Tactic **Obfuscated Files or Information** = ATT&CK Technique **Indicator Removal from Tools** = ATT&CK Sub-Technique **T1027 =** Technique Identifier **005** = Sub-Technique Identifier

In CAPA's final output, they referenced the MITRE Framework. This helps analysts or defenders map the file's behaviour to the adversary's playbook, which can help narrow down the scope of the investigation during an incident. To dig deeper into this topic, you might want to check out our room for MITRE ATT&CK Framework

MAEC Malware Attribute Enumeration and Characterization is a specialized language designed to encode and communicate complex details concerning malware. It contains an extensive range of attributes, including behaviours, artefacts, and interconnections among various instances of malware. This language functions as a standardized system for tracking and analyzing the complicated complexities of malware.

Malware Attribute Enumeration and Characterization

```
┌──────────────────────────┬──────────────────────────────────────────────────────────────────
──────────────┐
| MAEC Category            | MAEC Value
|
├──────────────────────────┼──────────────────────────────────────────────────────────────────
──────────────┤
| malware-category         | launcher
```

Let's check the table below to see the most commonly used MAEC values by CAPA: **Downloader** and **Launcher**.

MAEC Value	Description
Launcher	Exhibits behaviours that trigger specific actions similar to malware behaviour.
Downloader	Exhibits behaviours wherein it downloads and executes other files, usually seen on more complex malware.

When CAPA tags a file with a "**launcher**" MAEC value, it indicates that the file demonstrates behaviour similar to but not limited to:

- Dropping additional payloads
- Activating persistence mechanisms
- Connecting to command-and-control (C2) servers
- Executing specific functions

This is interesting! Some of these behaviours are also present in the Malware Behavior Catalogue (MBC) and Capability block, which we will discuss in the next task!

Additionally, when CAPA tags a file with a "**Downloader**" MAEC (malware attribute enumaration and characterization) value, it indicates that the file demonstrates behaviour similar but not limited to:

- Fetching additional payloads or resources from the internet
- pulling in updates
- executing secondary stages
- retrieving configuration files

Answer the questions below

> What is the sha256 of cryptbot.bin?
> ae7bc6b6f6ecb206a7b957e4bb86e0d11845c5b2d9f7a00a482bef63b567ce4c

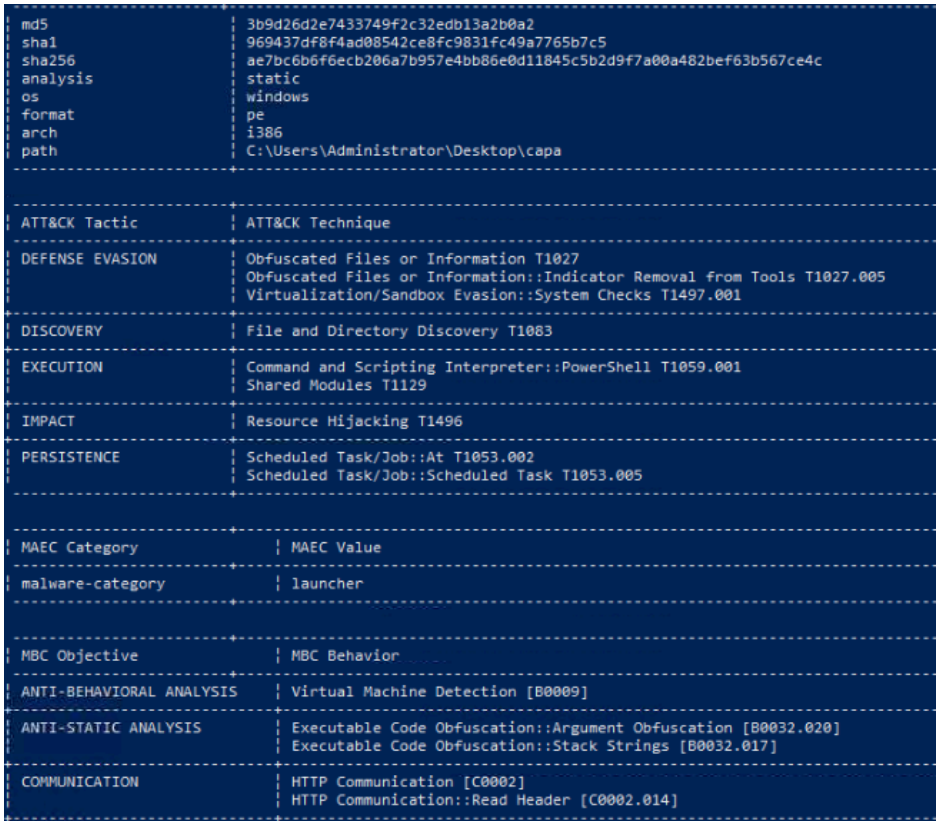
> What is the Technique Identifier of Obfuscated Files or Information? T1027

> What is the Sub-Technique Identifier of Obfuscated Files or

> Information::Indicator Removal from Tools? T1027.005

> When CAPA tags a file with this MAEC value, it indicates that it demonstrates behaviour similar to, but not limited to, Activating persistence mechanisms? launcher

> When CAPA tags a file with this MAEC value, it indicates that the file demonstrates behaviour similar to, but not limited to, Fetching additional payloads or resources from the internet? Downloader



md5	3b9d26d2e7433749f2c32edb13a2b0a2
sha1	969437df8f4ad08542ce8fc9831fc49a7765b7c5
sha256	ae7bc6b6f6ecb206a7b957e4bb86e0d11845c5b2d9f7a00a482bef63b567ce4c
analysis	static
os	windows
format	pe
arch	i386
path	C:\Users\Administrator\Desktop\capa

ATT&CK Tactic	ATT&CK Technique
DEFENSE EVASION	Obfuscated Files or Information T1027 Obfuscated Files or Information::Indicator Removal from Tools T1027.005 Virtualization/Sandbox Evasion::System Checks T1497.001
DISCOVERY	File and Directory Discovery T1083
EXECUTION	Command and Scripting Interpreter::PowerShell T1059.001 Shared Modules T1129
IMPACT	Resource Hijacking T1496
PERSISTENCE	Scheduled Task/Job::At T1053.002 Scheduled Task/Job::Scheduled Task T1053.005

MAEC Category	MAEC Value
malware-category	launcher

MBC Objective	MBC Behavior
ANTI-BEHAVIORAL ANALYSIS	Virtual Machine Detection [B0009]
ANTI-STATIC ANALYSIS	Executable Code Obfuscation::Argument Obfuscation [B0032.020] Executable Code Obfuscation::Stack Strings [B0032.017]
COMMUNICATION	HTTP Communication [C0002] HTTP Communication::Read Header [C0002.014]

Task 4 - Dissecting CAPA Results Part 2: Malware Behavior Catalogue

In this task, we will cover the following topics:

- MBC
- Objective
- Micro-Objective
- MBC Behaviors
- Micro-Behavior
- Methods

Let's move on to the next block. This terminal can also be seen from Task 2, where we ran the CAPA tool. Click on the view terminal below.

Malware Behavior Catalogue (MBC): MBC is designed to support various aspects of malware analysis, such as labelling, similarity analysis, and standardized reporting. Essentially, it serves as a catalogue of malware objectives and behaviours. MBC can also link to ATT&CK methods and log all behaviours and code features discovered during malware analysis. It's important to note that the names of MBC behaviours may or may not match the corresponding ATT&CK techniques. The information on behaviour pages complements the content on ATT&CK pages. In other words, when recording malware behaviours, MBC users will reference ATT&CK, but MBC does not duplicate ATT&CK information.

The content of MBC below can be represented in two formats.

Format	Sample	Explanation
OBJECTIVE::**Behavior**::**Method**[**Identifier**]	ANTI-STATIC ANALYSIS::Executable Code Obfuscation::Argument Obfuscation [B0032.020]	**Anti-static Analysis** = OBJECTIVE**Executable Code Obfuscation** = BEHAVIOR**Argument Obfuscation** = METHOD**BOO32.020** = IDENTIFIER
OBJECTIVE::**Behavior**::[**Identifier**]	COMMUNICATION::HTTP Communication:: [C0002]	**COMMUNICATION** = OBJECTIVE**HTTP Communication** = BEHAVIOR**C0002** = IDENTIFIER

The difference between the two formats is that the first format contains additional details called METHOD, which can also be coined as a sub-technique. We must also discuss the Objective, Behavior, and Methods to better understand this part.

Objective: The Objective are based on **ATT&CK tactics in the context of malware behaviour**, though not all are included. Furthermore, MBC has Anti-Behavioral and Anti-Static Analysis. These objectives are tailored for malware analysis with the use case of **characterizing malware**. See the table below for an explanation of each.

Objective	Explanation
Anti-Behavioral Analysis	Malware attempts to avoid detection by hindering behavioural analysis using tools like sandboxes or debuggers.
Anti-Static Analysis	Malware attempts to obstruct or add complexity to static analysis, making it more challenging for security professionals to identify and understand its malicious behaviours and intentions.
Collection	Malware focuses on identifying and gathering information from the targeted machine or network.
Command and Control	Malware typically establishes communication with compromised systems through various methods such as command and control servers, peer-to-peer networks, or other means. This communication allows the malware to control the compromised systems, enabling the attackers to execute commands, exfiltrate data, or carry out other malicious activities.
Credential Access	The primary aim of malware is to steal account credentials, such as usernames and passwords.
Defense Evasion	The malware aims to bypass and circumvent the various detection and security mechanisms present within the system to avoid being detected or mitigated.
Discovery	Malware Seeks to collect detailed information about the configuration and setup of the system or network environment, including hardware, software, and network infrastructure.
Execution	Malware is designed to execute unauthorized commands or code on a targeted computer system without the user's consent. This can include a wide range of harmful activities, such as stealing personal information, damaging files, or gaining unauthorized access to the system.
Exfiltration	Malware is designed to infiltrate computer systems or networks to steal and extract sensitive data. This can include personal information, financial details, and any other valuable data stored on the targeted system or network.
Impact	Malware aims to manipulate, disrupt, or damage computer systems and data. It can enter computers through infected emails, compromised websites, and other deceptive means, leading to financial loss, privacy breaches, and system instability.

Lateral Movement	Malware seeks to spread through the network, either actively through machine access or passively, such as via malicious emails.
Persistence	Malware is intentionally developed with the capability to remain undetected and operational on a computer system for an extended period.
Privilege Escalation	Malware seeks to infiltrate a computer system or network to gain elevated permissions or control. Once inside the target environment, malware can seek to escalate its privileges, access sensitive information, or take control of system resources for malicious purposes.

Micro-Objective: Micro-objectives are associated with micro-behaviors, which refer to an action or actions exhibited by potentially malicious software that isn't necessarily malicious and may serve various objectives. Example binaries are such as those in messaging apps. However, it's important to **note that these behaviours are typically abused.** That's why CAPA might have flagged this behaviour.

Micro-Objective	Description
PROCESS	exhibiting behaviours related to processes such as but not limited to Creating Process, Setting Thread Context, Terminating Process, and Checking Mutex.
MEMORY	exhibiting behaviours such as, but not limited to, Allocating Memory, Changing Memory Protection, and Freeing Memory.
COMMUNICATION	exhibiting behaviours such as (not limited to (DNS, FTP, HTTP, ICMP, SMTP) network traffic.
DATA	exhibiting behaviours such as but not limited to Checking strings, compressing, decoding and encoding data

The final output of CAPA, Objective, and Micro-Objective are shown only under the **Objective column**.

MBC Behaviors: The column MBC Behaviors contains behaviours and Micro-behaviors with or without its methods and identifiers. Please check the link MBC Summary(opens in new tab) for a listing of all MBC content. Below is a compiled version of Behaviors/Micro-behaviors and its Identifier.

Objective	Behavior	Identifiers	Explanation
ANTI-BEHAVIORAL ANALYSIS	**Lab Machine Detection**	**B0009**	The malware checks to see if it is running in a virtual environment. During its system reconnaissance, the malware examines various user and system artifacts.
ANTI-STATIC ANALYSIS	**Executable Code Obfuscation**	**B0032**	Executable code has been intentionally obscured to prevent static code analysis. This is a specific behavior related to the executable code of a malware sample, including its data and text sections.
EXECUTION	**Command and Scripting Interpreter**	**E1059**	Malware can exploit command and script interpreters to run malicious commands, scripts, or binaries. It targets built-in interpreters like cmd.exe or PowerShell on Windows or Bash on Unix-like systems. Attackers may also use other scripting languages like Python, Perl, or JavaScript.
DISCOVERY	**File and Directory Discovery**	**E1083**	Malware has the capability to search for specific files in particular locations by enumerating files and directories.
ANTI-STATIC ANALYSIS, DEFENSE EVASION	**Obfuscated Files or Information**	**E1027**	Malware can obfuscate files or information by encoding, encrypting, or otherwise, making them hard to analyze. It can also encode or encrypt malware samples itself.

Micro-Behavior: The term "low-level behaviors" in malware analysis refers to actions exhibited by malware that aren't necessarily malicious and may serve various objectives. These behaviors are often documented as "micro-behaviors" in the Malware Behavior Characteristics (MBC) analysis. Examples of such low-level behaviors include the creation of TCP sockets and evaluating specific conditions within strings. It's important

to **note that just because a behavior is categorized as low-level does not mean it is harmless,** as it may still be part of a larger malicious scheme.

Micro-Objective	Micro-Behaviors	Identifiers	Explanation
MEMORY	**Allocate Memory**	**C0007**	Malware frequently utilizes memory allocation as part of its strategy to unpack itself and execute its malicious activities.
PROCESS	**Create Process**	**C0017**	Malware creates a process via WMI or shellcode. It can also create a suspended process.
COMMUNICATION	**HTTP Communication**	**C0002**	Malware is capable of initiating HTTP communications.
DATA	**Check String**	**C0019**	Malware can inspect a string to identify specific characteristics, such as ASCII content, credit card numbers, and string length.
DATA	**Encode Data**	**C0026**	Malware has the capability to encode data using base64 and XOR.
FILE SYSTEM	**Create Directory**	**C0046**	Malware can create a directory.
FILE SYSTEM	**Delete File**	**C0047**	Malware has the capability to delete a file.
FILE SYSTEM	**Read File**	**C0051**	Malware can read a file.
FILE SYSTEM	**Writes File**	**C0052**	Malware has the capability to write to a file.

Note that in the final output of CAPA, Behavior and Micro-Behavior are shown only under the **Behavior column**.

Methods: Lastly, let's check the **METHODS**. Below are some methods included in the results from the previous sample. Methods are tied to behaviors; therefore, to fully see all methods, please refer to each specific behavior/micro behavior of interest.

Behavior	Methods or sub-technique	Identifier	Explanation
Executable Code Obfuscation	**Argument Obfuscation**	B0032.020	Simple number or string arguments to API calls are calculated at runtime, making analysis more difficult.
Executable Code Obfuscation	**Stack Strings**	B0032.017	Build and decrypt strings on the stack at each use, then discard to avoid obvious references.
HTTP Communication	**Read Header**	C0002.014	HTTP read header.
Encode Data	**Base64**	C0026.001	Malware may encode data using Base64.
Encode Data	**XOR**	C0026.002	Malware may use XOR to encode data.
Obfuscated Files or Information	**Encoding-Standard Algorithm**	E1027.m02	Encoding malware samples, files, or other information uses a standard algorithm (e.g., base64).

Now that we have a good overview and understanding of the MBC's contents, we should be able to explain the result of the previous sample. Therefore, let's do a quick recap using one of the results. Shall we?

Malware Behavior Catalogue

```
┌──────────────────────────┬──────────────────────────────────┐
│ MBC Objective            │ MBC Behavior                     │
├──────────────────────────┼──────────────────────────────────┤
│ DATA                     │ Encode Data::Base64 [C0026.001]  │
└──────────────────────────┴──────────────────────────────────┘
```

Here's the explanation for the above result. See the table below.

Label	Value	Explanation
MBC Objective	**DATA**	exhibiting behaviors such as, but not limited to, checking strings, compressing, decoding, and encoding data.
MBC Behavior	**Encode Data**	Malware has the capability to encode data using base64 and XOR.
Method	**Base64**	Malware may encode data using Base64.
Identifier	**C0026.001**	identifier relays information about a behavior. This also serves as a tag.

Knowing this information, you may simply say this file can use the base64 encoding scheme!

Answer the questions below

> What serves as a catalogue of malware objectives and behaviours? Malware Behavior Catalogue

> Which field is based on ATT&CK tactics in the context of malware behaviour? Objective

> What is the Identifier of "Create Process" micro-behavior? C0017

> What is the behaviour with an Identifier of B0009? Lab Machine Detection

> Malware can be used to obfuscate data using base64 and XOR. What is the related micro-behavior for this? Encode Data

> Which micro-behavior refers to "Malware is capable of initiating HTTP communications"? HTTP Communicatio

Task 5 - Dissecting CAPA Results Part 3: Namespaces

We will divide the discussion into two main topics: Capability and Namespace. In this task, we will focus on the discussion of **Namespace.** Below, you will find the `capa.exe` output. Note that this can also be viewed in Task 2. Click on the **View Terminal** below.

The content of this block is represented in the below format.

Format	Sample	Explanation
Capability(Rule Name)::**TLN(Top-Level Namespace)/Namespace**	reference anti-VM strings::Anti-Analysis/anti-vm/vm-detection	**Reference anti-VM strings** = Capability(Rule Name)**Anti-Analysis** = TLN or Top-Level Namespace**anti-vm/vm-detection** = Namespace

Namespaces: CAPA uses namespaces to group items with the same purpose.

Top-Level Namespace (TLN)	Explanation
anti-analysis	contains a set of rules specifically designed to detect behaviours exhibited by malware to evade analysis. These behaviours include obfuscation, packing, and anti-debugging techniques.
collection	contains a set of data-related rules that malware may enumerate and collect for exfiltration or other purposes. Think of it as the “data-gathering” aspect of malware behaviour.
communication	contains a set of rules that pertain to different communication behaviours demonstrated by malware. This encompasses how malware interacts with networks, including data transmission and reception, command and control communications, and other network-related behaviours.

Top-Level Namespace (TLN)	Explanation
compiler	contains a set of rules and configurations for recognizing specific build environments or compilers employed in generating executables. These namespaces essentially serve as the unique “signature” that identifies the compilation process of a program.
data-manipulation	contains a set of rules that govern the behaviours involved in altering data within executable files. This aspect can be considered the “data transformation” component of malware behaviour, encompassing actions such as String Encryption and Data Encoding.
executable	contains a set of rules pertaining to the attributes in executable files. These attributes include PE sections or debug info associated with the executable.
host-interaction	contains a set of rules related to behaviors involving interactions with the host system. This encompasses how malware interacts with its environment. Specifically, the rules in this namespace may capture behaviors related to reading, writing, or modifying files on disk, including creating, deleting, or modifying files and directories.”
impact	contains a set of rules related to the potential consequences or effects of a program’s behavior. Think of it as the aspect that focuses on the possible harm that this malware can cause. It may include behaviors related to establishing remote access, data exfiltration, destruction, or modification.”
internal	Rules contained within the system are not intended for direct use by analysts or for reporting. Instead, these rules are meant for internal purposes within the CAPA tool, serving as the behind-the-scenes aspect of rule development and execution.
lib	building blocks to create other rules
linking	contains rules to identify behaviors involving linking or dynamically loading external code or libraries during program execution. This is its primary function and is crucial for the program’s security. Understanding linking behavior is essential for several reasons. Malware often depends on external libraries or components (such as OpenSSL, Zlib, or other third-party libraries) to carry out specific tasks. Detecting these dependencies helps analysts understand the malware’s capabilities. External libraries also introduce an additional attack surface. If a vulnerability exists in a linked library, it can be exploited by the malware or defenders during analysis.
load-code	contains a set of rules and regulations related to the behaviors associated with dynamically loading or executing code during program execution. This concept can be equated to the “runtime code injection” aspect of malware behavior, which involves unauthorized code introduction during a program’s execution.
malware-family	contains a set of rules related to behaviors that are linked to particular malware families or groups. It serves as a

	way to identify the distinct characteristics or “signatures” associated with known malware families, allowing for more accurate detection and classification of potential threats.
nursery	this is a staging ground that contains rules for those who are not quite polished
persistence	contains rules related to behaviors associated with maintaining access or persistence within a compromised system. This namespace is essentially focused on understanding how malware can establish and maintain a presence within a compromised environment, allowing it to persist and carry out malicious activities over an extended period.
runtime	contains a set of rules that seek to identify the language or platform on which the program runs.
targeting	contains a set of rules related to behaviors exhibited by programs that interact with ATMs.

Top-Level Namespace (TLN)	Namespaces	Rule YAML File	Explanation
Anti-Analysis	anti-vm/vm-detection	reference-anti-vm-strings-targeting-virtualbox.ymlreference-anti-vm-strings-targeting-virtualpc.yml	“anti-vm/vm-detection” namespace contains rules to detect lab machine (VM) environments. These rules focus on identifying specific strings or patterns commonly used by malware to detect VMs while running. Using these rules, CAPA can identify if malware searches for VMware-specific registry keys, the presence of VMware tools, or other VM-related elements.
	obfuscation	obfuscated-with-dotfuscator.ymlobfuscated-with-smartassembly.yml	Malware often uses obfuscation techniques to make analysis more difficult. These include methods such as String Encryption, Code Obfuscation, Packing, and Anti-Debugging Tricks. The obfuscation namespace addresses these techniques, which conceal or obscure the true purpose of the code.

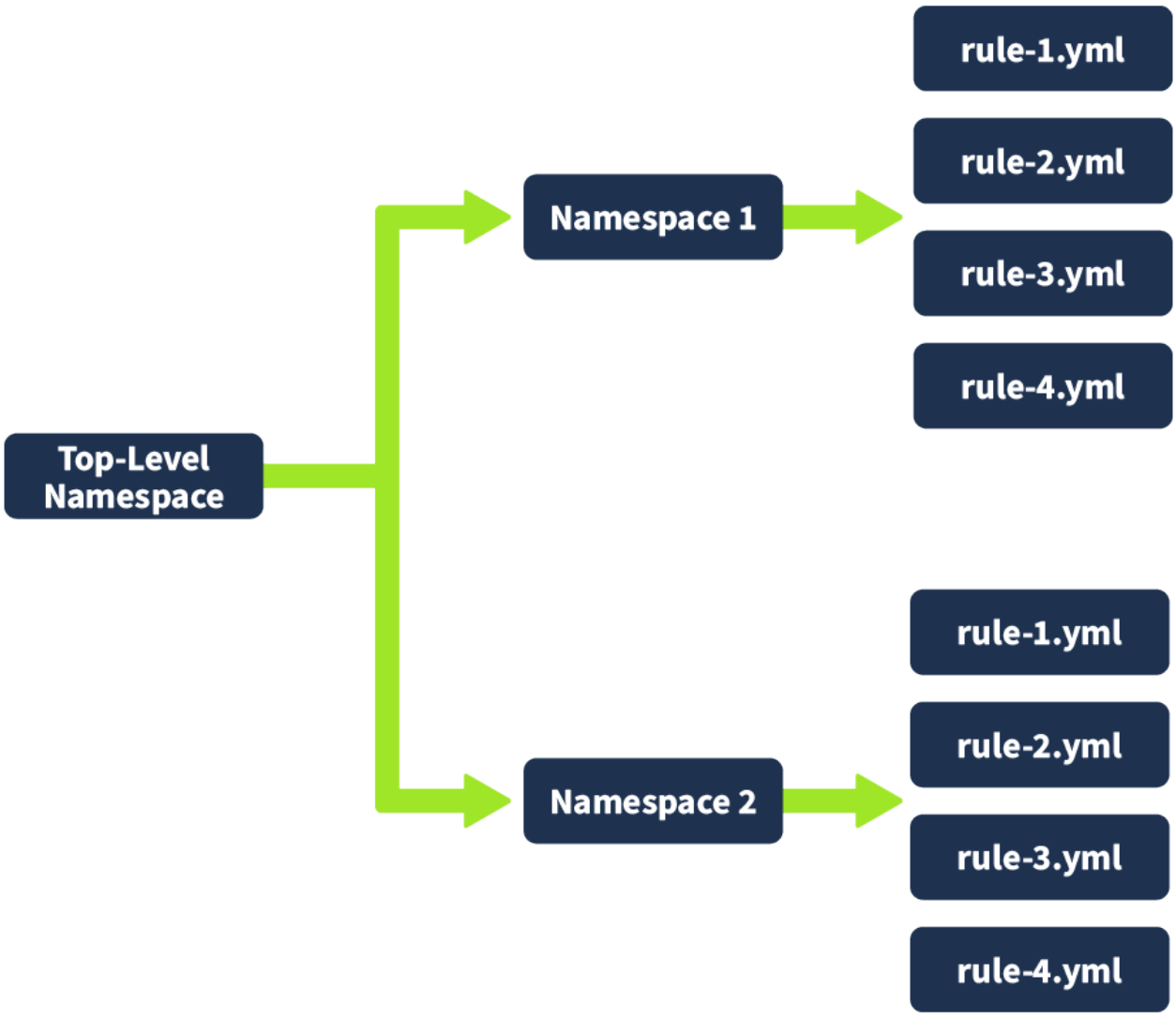
For this, we only used **Anti-Analysis** as the TLN or Top-Level Namespace. Under this TLN, we have grouped namespaces, such as **anti-vm/vm-detection** and **obfuscation**. Each namespace has a collection of rules inside them that are also grouped together. For **anti-vm/vm-detection,** we have rules, and it's config file, such as:

- **reference-anti-vmstrings-targeting-virtualbox.yml**
- **reference-anti-vmstrings-targeting-virtualpc.yml**

The same goes for the **obfuscation** namespace. We have rules that are grouped, such as:

- **obfuscated-with-dotfuscator.yml**
- **obfuscated-with-smartassembly.yml**

Again, do note that this is still under TLN **Anti-Analysis**!



In addition to what was mentioned in the above table, there are still a few more namespaces under **Anti-Analysis** with corresponding rules. If you want to dig deeper, please check this link(opens in new tab). Use this link(opens in new tab) if you are interested in the other TLN or Top-Level Namespaces, such as collection, compiler, persistence, linking, and impact.

Answer the questions below

> Which top-level Namespace contains a set of rules specifically designed to detect behaviours, including obfuscation, packing, and anti-debugging techniques exhibited by malware to evade analysis? anti-analysis

> Which Top-Level Namespace contains rules related to behaviours associated with maintaining access or persistence within a compromised system? This namespace is focused on understanding how malware can establish and maintain a presence within a compromised environment, allowing it to persist and carry out malicious activities over an extended period. persistence

> Which namespace addresses techniques such as String Encryption, Code Obfuscation, Packing, and Anti-Debugging Tricks, which conceal or obscure the true purpose of the code? obfuscation

> Which Top-Level Namespace Is a staging ground for rules that are not quite polished? Nursery

> Proceed to the next task for the 2nd part of the discussion! No answer needed

Task 6 - Dissecting CAPA Results Part 4: Capability

In this task, we will continue the discussion from the previous task.

Capability: Below is a table with the **Capability and its related TLN, namespace, and the rules associated** with the yaml file. Please have a good look.

Capability	Top-Level Namespace (TLN)	Namespaces	Rule YAML file	Notes
reference anti-VM strings	**Anti-Analysis**(opens in new tab)	anti-vm/vm-detection	reference-anti-vm-strings.yml	To check all rules under this namespace, click here(opens in new tab)
reference anti-VM strings targeting VMWare	**Anti-Analysis**	anti-vm/vm-detection	reference-anti-vm-strings-targeting-vmware.yml	To check all rules under this namespace, click here(opens in new tab)
reference anti-VM strings targeting VirtualBox	**Anti-Analysis**	anti-vm/vm-detection	reference-anti-vm-strings-targeting-virtualbox.yml	You may check the TLN(Top-Level Namespace).
reference HTTP User-Agent string	**Communication**(opens in new tab)	http/client	reference-http-user-agent-string.yml	To check all rules under this namespace, click here(opens in new tab))
check HTTP status code	**Communication**	http	check-http-status-code.yml	To check all rules under this namespace, click here(opens in new tab)
reference Base64 string	**Data Manipulation**(opens in new tab)	encoding/base64	reference-base64-string.yml	To check all rules under this namespace, click here(opens in new tab)
encode data using XOR	**Data Manipulation**	encoding/XOR	encode-data-using-xor.yml	To check all rules under this namespace, click here(opens in new tab)
contain a thread local storage (.tls) section	**Executable**(opens in new tab)	pe/section/tls	contain-a-thread-local-storage-tls-section.yml	You may check the TLN(Top-Level Namespace) for more rules.
get common file path	**Host-Interaction**(opens in new tab)	file-system	get-common-file-path.yml	You may check the TLN(Top-Level Namespace) for more rules.
create directory	**Host-Interaction**	file-system/create	create-directory.yml	You may check the TLN(Top-

				Level Namespace) for more rules.
delete file	**Host-Interaction**	file-system/delete	delete-file.yml	To check all rules under this namespace, click here(opens in new tab)
read file on Windows	**Host-Interaction**	file-system/read	read-file-on-windows.yml	To check all rules under this namespace, click here(opens in new tab)
write file on Windows	**Host-Interaction**	file-system/write	write-file-on-windows.yml	To check all rules under this namespace, click here(opens in new tab)
get thread local storage value	**Host-Interaction**	process	get-thread-local-storage-value.yml	This rule is found under **TLN Nursery(opens in new tab),** a staging ground for unpolished rules.
allocate or change RWX memory	**Host-Interaction**	process/inject	allocate-or-change-rwx-memory.yml	To check all rules under this namespace, click here(opens in new tab)
create process on Windows	**Host-Interaction**	process create	create-process-on-windows.yml	To check all rules under this namespace, click here(opens in new tab)
reference cryptocurrency strings	**Impact**(opens in new tab)	impact/cryptocurrency	reference-cryptocurrency-strings.yml	This rule is found under **TLN Nursery(opens in new tab),** a staging ground for unpolished rules.
link function at runtime on Windows	**Linking**(opens in new tab)	runtime-linking	link-function-at-runtime-on-windows.yml	To check all rules under this namespace, click here(opens in new tab)
parse PE header	**load-code**(opens in new tab)	load-code/pe	parse-pe-header.ymlresolve-function-by-parsing-pe-exports.yml	To check all rules under this namespace, click here(opens in new tab)
resolve function by parsing PE exports	**load-code**(opens in new tab)	load-code/pe	resolve-function-by-parsing-pe-exports.yml	To check all rules under this namespace, click here(opens in new tab)
run PowerShell expression	**load-code**(opens in new tab)	load-code/PowerShell	run-powershell-expression.yml	To check all rules under this namespace, click here(opens in new tab)
schedule task via at	**persistence**(opens in new tab)	scheduled-tasks	schedule-task-via-at.yml	You may check the TLN(Top-

				Level Namespace) for more rules.
schedule task via schtasks	**persistence**(opens in new tab)	scheduled-tasks	schedule-task-via-schtasks.yml	You may check the TLN(Top-Level Namespace) for more rules.

To further explain this, let's check the first capability on the table, **"reference anti-VM strings"**.

- We note that the related rules in YML format are **reference-anti-vmstrings.yml**
- This is under the namespace **anti-vm/vmdetection**
- which is also under the Top-Level Namespace **Anti-Analysis**

This tells us that CAPA was able to identify that the potentially malicious software **searches for VMware-specific registry keys**, the **presence of VMware tools**, or other **VM-related elements** by using the **reference-anti-vm-strings.yml** rule yaml file. Malware typically does this behaviour to avoid detection. That is why CAPA flagged this one.

Let's have another example. Let's look at "***schedule task via schtasks***".

- We note that the related rules in YML format is **schedule-task-via-schtasks.yml**
- This is under the namespace **scheduled-tasks**
- which is also under the Top-Level Namespace **persistence**

This tells us that CAPA could identify behaviours related to scheduled tasks within the Windows operating system. It might have recognized patterns indicating that the executable registers itself as a scheduled task to maintain persistence using the rule defined in **schedule-task-via-schtasks.yml.**

That's right! The item under Capability has the same name as the YML files under the Rules, with the addition of a dash (-) character between spaces! Simple because Capability is the name of the rule.

Now, we want to take note of some exceptions here. Where the Capability or rules are not located under its Namespace, let's take the Capability **reference cryptocurrency strings** from the table above; this should be under the **Impact** Top-Level Namespace, right? However, if you go through the folders, you won't be able to find the corresponding rules. It will be located under the **Nursery** TLN. This is the placeholder for rules that are not quite polished yet.

Now that we have a good overview and understanding of the Capability and Namespace contents, we should be able to explain the sample result from the previous tasks. Therefore, let's do a quick recap using one of the results. Shall we?

Malware Behavior Catalogue

```
┌──────────────────────────────────────────┬──────────────────────────────────────────┐
│ Capability                               │ Namespace                                │
├──────────────────────────────────────────┼──────────────────────────────────────────┤
│ reference Base64 string                  │ data-manipulation/encoding/base64        │
└──────────────────────────────────────────┴──────────────────────────────────────────┘
```

Here's the explanation for the above result. See the table below.

Label	Value	Explanation
Capability	**reference base64 string**	Malware has the capability to encode data using a base64 scheme.
Top-Level Namespace	**data-manipulation**	contains a set of rules that govern the behaviors involved in altering data within executable files. This aspect can be considered the “data transformation” component of malware behaviour, encompassing actions such as String Encryption and Data Encoding.
Namespace	**encoding/base64**	this namespace consists of rules for encoding and decoding data using Base64 and XOR
Rule YAML File Matched?	**reference-base64-string.yml**	Remember that the capability's name is also the rule's name with an additional dash (-) character between spaces.

Answer the questions below

> What rule yaml file was matched if the Capability or rule name is check HTTP status code? check-http-status-code.yml

> What is the name of the Capability if the rule YAML file is reference-anti-vm-strings.yml? reference anti-VM strings

> Which TLN or Top-Level Namespace includes the Capability or rule name run PowerShell expression? load-code

> Check the conditions inside the check-for-windows-sandbox-via-registry.yml rule file from this link (opens in new tab). What is the value of the API that ends in Ex is it looking for? RegOpenKeyEx

Task 7 - More Information, more fun!

In this task, we will seek to determine the reason for triggering the rules and the conditions involved. We will use the parameter

```
-vv
```

or **very verbose** to achieve this.

Option	Description	Sample Syntax
`-v` or `--verbose`	enable verbose result document	`capa.exe -v .\cryptbot.bin`
`-vv` or `--vverbose`	enable very verbose result document)	`capa.exe -vv .\cryptbot.bin`

```
PS C:\Users\Administrator\Desktop\capa> capa -vv .\cryptbot.bin
loading : 100%|████████████████████| 485/485 [00:00<00:00, 1108.84     rules/s]
/ analyzing program...
```

This will give us more detailed results; however, this will take a lot of time. We already processed this and named the file `cryptbot_vv.txt` in "`C:\Users\Administrator\Desktop\capa`". Remember, you don't need to wait for the tool to finish running; we did this so that you can get a feel for running the tool and experiment with its parameters. Open another PowerShell terminal and look inside the file using the command `Get-Content cryptbot_vv.txt`.

```
PS C:\Users\Administrator\Desktop\capa> Get-Content .\cryptbot_vv.txt
```

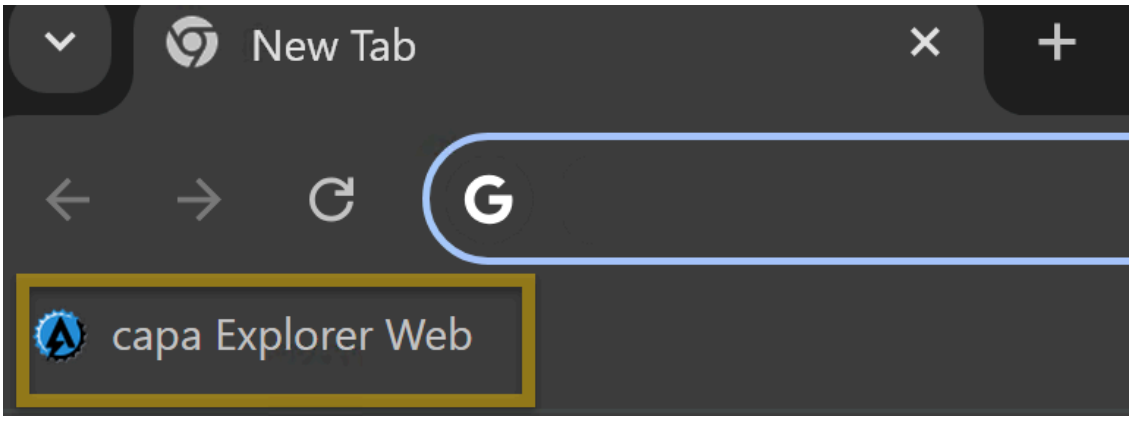
So, have you opened the file? Do you have more than three thousand lines similar to this one? Accessing this vast amount of information using a terminal or a text editor will be challenging. To analyze this result with ease, we need to do two things. First, we will use the parameter `-j` and `-vv`, and direct the result to a **.json** file. The command would be `capa.bin -j -vv .\cryptbot.bin > cryptbot_vv.json`

```
PS C:\Users\Administrator\Desktop\capa> capa.bin -j -vv .\cryptbot.bin > cryptbot_vv.
json
loading : 100%|██████████████████| 485/485 [00:00<00:00, 1108.84     rules/s]
/ analyzing program...
```

Again, this will take a lot of time, similar to the previous commands we run. We have processed this and created a file named **cryptbot_vv.json** located in "**C:\Users\Administrator\Desktop\capa**". Additionally, we've attached the file to this task. Remember, you don't need to wait for the tool to finish running; we did this so that you can get a feel for running the tool and experiment with its parameters.

CAPA Web Explorer

The second thing we need to do is upload the file to **CAPA Explorer Web**. We can either use their online version here on this link(opens in new tab), or the offline version already in the lab machine. There is a Google Chrome browser on the desktop named **capa_web_explorer_offline.html**. Alternatively, you can access this using a regular Chrome browser bookmark. Note that the local page may take up to a minute to load in Chrome on the lab machine.



Now, we should have access to the homepage!

capa: identify program capabilities

capa Explorer Web is a web-based tool to explore the capabilities identified by capa. This tool allows you to interactively browse and display capa results in multiple viewing modes.

New to capa? Follow these quick steps to get started:

1. Install capa , e.g.
   - download the latest standalone executable release
   - or run `$ pip install flare-capa`
2. Analyze a sample and save the JSON results:
   - `$ capa -j /path/to/file > result.json`
3. Load the JSON results file into capa Explorer Web

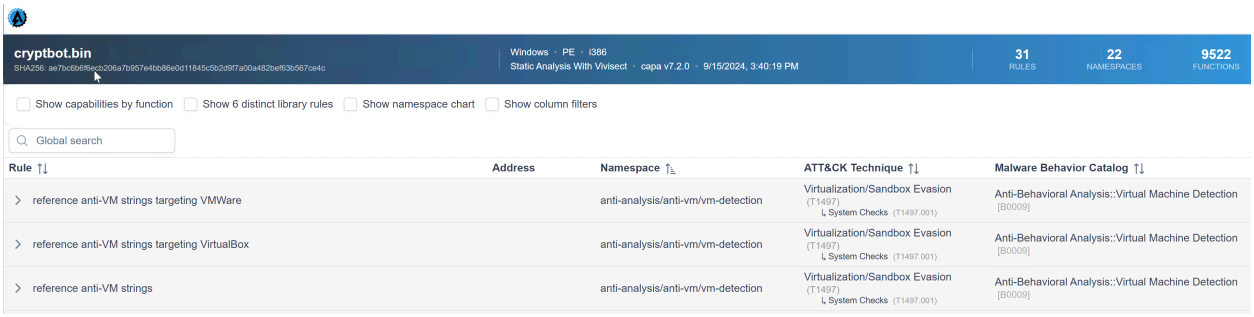
For more detailed information, explore the capa GitHub repository .

You can download capa Explorer Web for offline usage via the download button in the top-right corner of this page.

Upload from local OR Load from URL

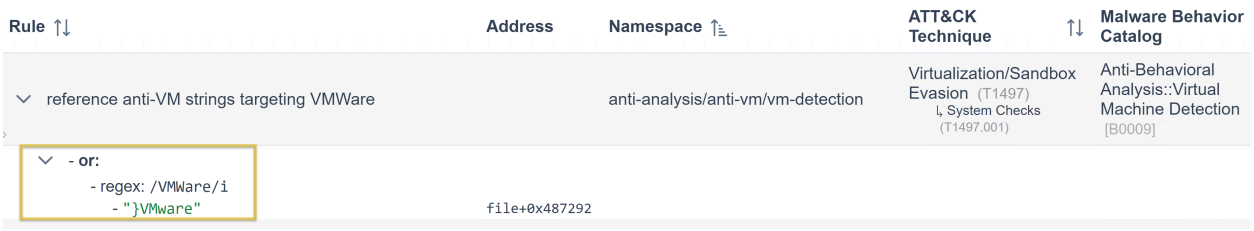
Look for the button **Upload from local** located at the bottom left of the page and select the file **cryptbot_vv.json** in **C:\Users**

Administrator\Desktop\capa . Once uploaded, you should have a similar output.



Doesn't this look good and easier to use? I bet it is! Now, it's time to explore this excellent addition to the tool. We'll review some capabilities and check what precisely within the rule was matched. This will give us a better idea of how the rule works.

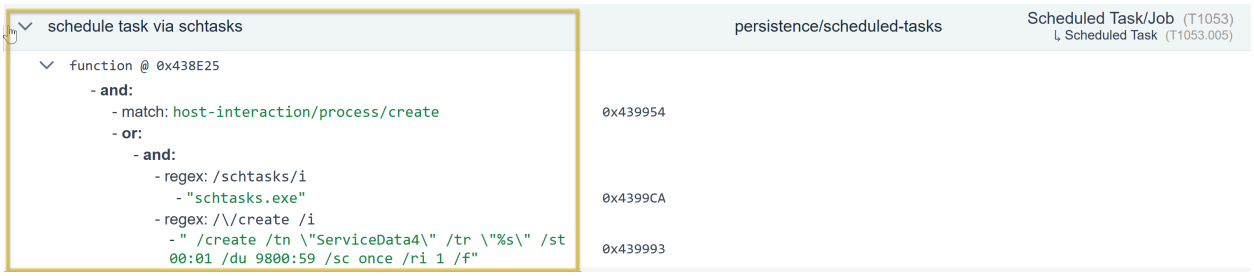
Let's go through our first example below. We know that the capability was to **reference anti-VM strings targeting VMWare,** and the corresponding rule config file or yaml file is **anti-VM-Strings-targeting-VMWare.yml.** Notice the box from the image.



Then, let us show you an overview of the rule's content. Focus on the **features** as CAPA uses the succeeding strings below to check if strings are within the analyzed file.

Did you see it? That's right! Under the features, the "**string: /VMWare/i"** is being referenced by CAPA Web Explorer. Simply, CAPA is saying that under this namespace, we could identify strings with a value of **VMWare** by using the conditions within the rule and with regex**.**

Let's have another sample. We know that the capability was to reference the **scheduled task via schtasks**, and the corresponding rule was to schedule the **task via schtasks.yml.** Notice the box from the image.



The same applies to our first example; we will show you the rule's content overview. Focus on the features, as CAPA uses the succeeding strings below to check if there are strings within the file being analyzed.

Under the feature, the " **string: /schtasks/i** and **/\/create /i"** is referenced by CAPA Web Explorer. Simply, CAPA is saying under this namespace , and by using the conditions within the rule and with regex, we could identify strings with a value of **schtasks** and **create**.

Global Search Box: Another cool feature of this tool is its filter options and the Global Search box, which are very helpful.

We could quickly examine this overwhelming information using CAPA Web Explorer compared to any text editor.

Task 8 - Conclusion

In this room, we discussed how CAPA is leveraged and plays a critical role in cyber security by analyzing potentially malicious or dangerous software and proactively searching for potential threats using static analysis. It achieves this by automating the process of detecting intricate functionalities within executable files and presenting the findings in an easily understandable format for security experts. This simplification of complex reverse engineering concepts aids in swiftly comprehending potentially harmful software, ultimately

strengthening incident response and defensive strategies.